

Complete AI Agents Cheat Sheet

A comprehensive guide to building production-ready AI agents with LangChain, LangGraph, LangSmith, and related tools

Table of Contents

- [1. Core Concepts](#)
- [2. LangChain Fundamentals](#)
- [3. LangGraph - Advanced Agent Workflows](#)
- [4. LangSmith - Monitoring & Debugging](#)
- [5. Agent Architectures](#)
- [6. Memory Systems](#)
- [7. Tools & Function Calling](#)
- [8. Retrieval Systems \(RAG\)](#)
- [9. Evaluation & Testing](#)
- [10. Production Deployment](#)
- [11. Best Practices](#)

Core Concepts

What is an AI Agent?

An AI agent is an autonomous system that:

- Perceives** its environment through inputs
- Reasons** about what actions to take
- Acts** using tools and APIs
- Learns** from feedback and experience

Agent Types

Type	Description	Use Case
ReAct	Reasoning + Acting in iterative loops	General problem-solving
Plan-and-Execute	Plans first, then executes steps	Complex multi-step tasks
Reflexion	Self-reflects and improves	Learning from failures

Type	Description	Use Case
Tool-using	Calls external APIs/functions	Data retrieval, calculations
Conversational	Maintains context in dialogue	Chatbots, assistants
Multi-agent	Multiple agents collaborate	Complex workflows

LangChain Fundamentals

Installation

```
bash

# Core package
pip install langchain

# Community integrations
pip install langchain-community

# Specific integrations
pip install langchain-openai
pip install langchain-anthropic
pip install langchain-google-genai
pip install langchain-cohere

# Vector stores
pip install langchain-chroma
pip install langchain-pinecone
pip install langchain-weaviate
```

Core Components

1. Language Models

```
python
```

```
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
from langchain_google_genai import ChatGoogleGenerativeAI

# Initialize LLM
llm = ChatOpenAI(
    model="gpt-4",
    temperature=0.7,
    max_tokens=1000,
    timeout=30,
    max_retries=2
)

# Streaming
for chunk in llm.stream("Tell me a joke"):
    print(chunk.content, end="", flush=True)

# Async
response = await llm.ainvoke("What is AI?")
```

2. Prompts & Prompt Templates

```
python
```

```

from langchain.prompts import (
    PromptTemplate,
    ChatPromptTemplate,
    MessagesPlaceholder,
    FewShotPromptTemplate
)

# Simple template
prompt = PromptTemplate(
    input_variables=["topic"],
    template="Write a poem about {topic}"
)

# Chat template
chat_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful AI assistant."),
    ("human", "{input}"),
    MessagesPlaceholder(variable_name="agent_scratchpad")
])

# Few-shot template
examples = [
    {"input": "happy", "output": "joyful"},
    {"input": "sad", "output": "melancholic"}
]

few_shot_prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=PromptTemplate(
        input_variables=["input", "output"],
        template="Input: {input}\nOutput: {output}"
    ),
    prefix="Translate emotions to synonyms:",
    suffix="Input: {input}\nOutput:",
    input_variables=["input"]
)

```

3. Chains (LCEL - LangChain Expression Language)

python

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough
```

```
# Basic chain
```

```
chain = prompt | llm | StrOutputParser()
result = chain.invoke({"topic": "AI"})
```

```
# Chain with multiple steps
```

```
chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)
```

```
# Parallel chains
```

```
from langchain_core.runnables import RunnableParallel
```

```
chain = RunnableParallel(
    summary=summary_chain,
    keywords=keyword_chain
)
```

```
# Conditional routing
```

```
from langchain_core.runnables import RunnableBranch
```

```
branch = RunnableBranch(
    (lambda x: "code" in x["input"], code_chain),
    (lambda x: "math" in x["input"], math_chain),
    general_chain # default
)
```

4. Output Parsers

```
python
```

```
from langchain.output_parsers import (
    PydanticOutputParser,
    StructuredOutputParser,
    CommaSeparatedListOutputParser
)
from pydantic import BaseModel, Field

# Pydantic parser
class Person(BaseModel):
    name: str = Field(description="Person's name")
    age: int = Field(description="Person's age")

parser = PydanticOutputParser(pydantic_object=Person)

prompt = PromptTemplate(
    template="Extract person info.\n{format_instructions}\n{query}",
    input_variables=["query"],
    partial_variables={"format_instructions": parser.get_format_instructions()}
)

# Structured parser
response_schemas = [
    ResponseSchema(name="answer", description="answer to question"),
    ResponseSchema(name="confidence", description="confidence score 0-1")
]
parser = StructuredOutputParser.from_response_schemas(response_schemas)
```

LangGraph

Installation

```
bash

pip install langgraph
pip install langgraph-checkpoint-sqlite # For persistence
```

Core Concepts

LangGraph enables building **stateful, cyclic workflows** with:

- **Nodes:** Functions that process state
- **Edges:** Connections between nodes
- **State:** Shared data structure
- **Conditional edges:** Dynamic routing

- **Persistence:** Save/resume workflows

Basic Graph Structure

python

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, Annotated
import operator

# Define state
class AgentState(TypedDict):
    messages: Annotated[list, operator.add]
    next_action: str
    iterations: int

# Create graph
workflow = StateGraph(AgentState)

# Add nodes
def agent_node(state: AgentState):
    # Your logic here
    return {
        "messages": [AIMessage(content="Response")],
        "iterations": state["iterations"] + 1
    }

def tool_node(state: AgentState):
    # Execute tools
    return {"messages": [ToolMessage(content="Result")]}

workflow.add_node("agent", agent_node)
workflow.add_node("tools", tool_node)

# Add edges
workflow.set_entry_point("agent")

# Conditional edge
def should_continue(state: AgentState):
    if state["iterations"] >= 5:
        return "end"
    return "continue"

workflow.add_conditional_edges(
    "agent",
    should_continue,
    {
        "continue": "tools",
        "end": END
    }
)
```

```
workflow.add_edge("tools", "agent")
```

```
# Compile
```

```
app = workflow.compile()
```

Advanced Graph Patterns

1. ReAct Agent with LangGraph

```
python
```

```
from langgraph.prebuilt import create_react_agent
```

```
from langchain_core.tools import tool
```

```
@tool
```

```
def search(query: str) -> str:
```

```
    """Search the web for information."""
```

```
    return f'Results for: {query}'
```

```
@tool
```

```
def calculator(expression: str) -> str:
```

```
    """Evaluate mathematical expressions."""
```

```
    return str(eval(expression))
```

```
tools = [search, calculator]
```

```
agent = create_react_agent(llm, tools)
```

```
# Run agent
```

```
for chunk in agent.stream(
```

```
    {"messages": [("human", "What's 25 * 4?)]},
```

```
    stream_mode="values"
```

```
):
```

```
    chunk["messages"][-1].pretty_print()
```

2. Multi-Agent System

```
python
```

```

from langgraph.graph import StateGraph, END

class MultiAgentState(TypedDict):
    messages: Annotated[list, operator.add]
    next_agent: str
    final_answer: str

# Define specialized agents
def researcher(state):
    response = research_llm.invoke(state["messages"])
    return {
        "messages": [response],
        "next_agent": "writer"
    }

def writer(state):
    response = writer_llm.invoke(state["messages"])
    return {
        "messages": [response],
        "next_agent": "critic"
    }

def critic(state):
    response = critic_llm.invoke(state["messages"])
    if "approved" in response.content.lower():
        return {"final_answer": response.content, "next_agent": END}
    return {"messages": [response], "next_agent": "writer"}

# Build graph
graph = StateGraph(MultiAgentState)
graph.add_node("researcher", researcher)
graph.add_node("writer", writer)
graph.add_node("critic", critic)

graph.set_entry_point("researcher")
graph.add_edge("researcher", "writer")
graph.add_edge("writer", "critic")
graph.add_conditional_edges(
    "critic",
    lambda x: x["next_agent"],
    {"writer": "writer", END: END}
)

app = graph.compile()

```

3. Human-in-the-Loop

```
python

from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import StateGraph

memory = MemorySaver()

def human_approval_node(state):
    # Pause for human input
    return state

workflow.add_node("human_approval", human_approval_node)
workflow.add_edge("agent", "human_approval")
workflow.add_edge("human_approval", "execute")

app = workflow.compile(
    checkpointer=memory,
    interrupt_before=["human_approval"]
)

# Run until interrupt
config = {"configurable": {"thread_id": "1"}}
for event in app.stream({"messages": [("human", "Task")]}), config):
    print(event)

# Resume after human input
app.update_state(
    config,
    {"approved": True},
    as_node="human_approval"
)

# Continue
for event in app.stream(None, config):
    print(event)
```

4. Persistence & Checkpointing

```
python
```

```
from langgraph.checkpoint.sqlite import SqliteSaver

# SQLite persistence
with SqliteSaver.from_conn_string("checkpoints.db") as checkpointer:
    app = workflow.compile(checkpointer=checkpointer)

    config = {"configurable": {"thread_id": "user-123"}}

    # Run and save state
    app.invoke({"messages": [("human", "Hello")]}, config)

    # Resume later
    app.invoke({"messages": [("human", "Continue")]}, config)

    # Get state history
    for state in app.get_state_history(config):
        print(state)
```

LangGraph Built-in Agents

```
python

from langgraph.prebuilt import create_react_agent, ToolNode

# Quick ReAct agent
agent = create_react_agent(llm, tools)

# Custom with ToolNode
workflow = StateGraph(AgentState)
workflow.add_node("agent", agent_node)
workflow.add_node("tools", ToolNode(tools))
```

LangSmith

Setup

```
bash

pip install langsmith

# Set environment variables
export LANGCHAIN_TRACING_V2=true
export LANGCHAIN_API_KEY=your_api_key
export LANGCHAIN_PROJECT=my-project
```

Tracing & Monitoring

```
python

from langsmith import Client
from langchain.callbacks import LangChainTracer

# Initialize client
client = Client()

# Automatic tracing (just run your chains)
chain = prompt | llm
result = chain.invoke({"input": "Hello"})

# Manual tracing
from langsmith import traceable

@traceable(run_type="chain", name="my_custom_chain")
def my_function(input_text):
    response = llm.invoke(input_text)
    return response

# Add metadata and tags
chain.invoke(
    {"input": "Hello"},
    config={
        "metadata": {"user_id": "123", "environment": "prod"},
        "tags": ["production", "customer-support"]
    }
)
```

Evaluation & Testing

```
python
```

```

from langsmith import evaluate

# Create dataset
examples = [
    ("What is AI?", "AI is artificial intelligence..."),
    ("Explain ML", "Machine learning is...")
]

dataset_name = "qa-dataset"
client.create_dataset(dataset_name, description="QA pairs")

for input_text, output_text in examples:
    client.create_example(
        inputs={"question": input_text},
        outputs={"answer": output_text},
        dataset_name=dataset_name
    )

# Evaluator function
def qa_evaluator(run, example):
    prediction = run.outputs["answer"]
    reference = example.outputs["answer"]

    # Custom evaluation logic
    score = similarity(prediction, reference)

    return {"score": score}

# Run evaluation
results = evaluate(
    lambda inputs: chain.invoke(inputs),
    data=dataset_name,
    evaluators=[qa_evaluator],
    experiment_prefix="qa-eval"
)

# Built-in evaluators
from langchain.evaluation import load_evaluator

criteria_evaluator = load_evaluator("criteria", criteria="conciseness")
string_distance = load_evaluator("string_distance")
embedding_distance = load_evaluator("embedding_distance")

```

Online Evaluation

python

```
from langsmith import evaluate_existing

# Evaluate production runs
evaluate_existing(
    project_name="production-runs",
    evaluators=[qa_evaluator],
    filters='eq(metadata.environment, "prod")'
)
```

Feedback Collection

```
python

# Submit feedback
client.create_feedback(
    run_id="run-uuid",
    key="user-rating",
    score=0.9,
    comment="Great response!"
)

# Query feedback
feedback = client.list_feedback(run_ids=["run-uuid"])
```

Datasets & Few-shot Examples

```
python

# Upload dataset from file
import json

with open("examples.json") as f:
    examples = json.load(f)

dataset = client.create_dataset("my-dataset")
for ex in examples:
    client.create_example(
        inputs=ex["input"],
        outputs=ex["output"],
        dataset_id=dataset.id
    )

# Use for few-shot learning
examples = client.list_examples(dataset_name="my-dataset")
```

Agent Architectures

1. ReAct (Reasoning + Acting)

```
python

from langchain.agents import create_react_agent, AgentExecutor
from langchain import hub

# Get ReAct prompt
prompt = hub.pull("hwchase17/react")

agent = create_react_agent(llm, tools, prompt)

agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True,
    max_iterations=10,
    handle_parsing_errors=True
)

result = agent_executor.invoke({"input": "What's the weather in Paris?"})
```

2. Plan-and-Execute

```
python

from langchain.agents import create_plan_and_execute_agent

agent = create_plan_and_execute_agent(
    llm=llm,
    tools=tools,
    verbose=True
)

# Agent first creates a plan, then executes each step
result = agent.invoke({"input": "Research AI trends and write a report"})
```

3. Structured Chat Agent

```
python
```

```
from langchain.agents import create_structured_chat_agent

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant"),
    MessagesPlaceholder(variable_name="chat_history"),
    ("human", "{input}"),
    MessagesPlaceholder(variable_name="agent_scratchpad")
])

agent = create_structured_chat_agent(llm, tools, prompt)
```

4. OpenAI Functions Agent

```
python

from langchain.agents import create_openai_functions_agent

agent = create_openai_functions_agent(llm, tools, prompt)

agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    return_intermediate_steps=True
)
```

5. Custom Agent with LangGraph

```
python
```

```
from langgraph.graph import StateGraph, END
from langchain_core.messages import HumanMessage, AIMessage
```

```
class CustomAgentState(TypedDict):
    messages: Annotated[list, operator.add]
    plan: list[str]
    current_step: int
    completed: bool
```

```
def planner(state):
    """Create execution plan"""
    messages = state["messages"]
    response = planner_llm.invoke([
        SystemMessage(content="Create a step-by-step plan"),
        *messages
    ])
    plan = response.content.split("\n")
    return {"plan": plan, "current_step": 0}
```

```
def executor(state):
    """Execute current step"""
    step = state["plan"][state["current_step"]]
    response = executor_llm.invoke([
        SystemMessage(content=f"Execute: {step}"),
        *state["messages"]
    ])
    return {
        "messages": [AIMessage(content=response.content)],
        "current_step": state["current_step"] + 1
    }
```

```
def should_continue(state):
    if state["current_step"] >= len(state["plan"]):
        return "end"
    return "continue"
```

```
workflow = StateGraph(CustomAgentState)
workflow.add_node("planner", planner)
workflow.add_node("executor", executor)
workflow.set_entry_point("planner")
workflow.add_edge("planner", "executor")
workflow.add_conditional_edges(
    "executor",
    should_continue,
    {"continue": "executor", "end": END}
)
```

```
agent = workflow.compile()
```

Memory Systems

1. Conversation Buffer Memory

```
python

from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory(
    memory_key="chat_history",
    return_messages=True
)

# Add to conversation
memory.save_context(
    {"input": "Hello"},
    {"output": "Hi! How can I help?"}
)

# Load memory
history = memory.load_memory_variables({})
```

2. Conversation Buffer Window Memory

```
python

from langchain.memory import ConversationBufferWindowMemory

# Keep only last k interactions
memory = ConversationBufferWindowMemory(
    k=5, # Last 5 exchanges
    return_messages=True
)
```

3. Conversation Summary Memory

```
python
```

```
from langchain.memory import ConversationSummaryMemory

memory = ConversationSummaryMemory(
    llm=llm,
    return_messages=True
)

# Automatically summarizes conversation
```

4. Conversation Summary Buffer Memory

```
python

from langchain.memory import ConversationSummaryBufferMemory

memory = ConversationSummaryBufferMemory(
    llm=llm,
    max_token_limit=1000, # Summarize when exceeds
    return_messages=True
)
```

5. Vector Store-Backed Memory

```
python

from langchain.memory import VectorStoreRetrieverMemory
from langchain_community.vectorstores import Chroma

vectorstore = Chroma(embedding_function=embeddings)

retriever = vectorstore.as_retriever(search_kwargs={"k": 5})

memory = VectorStoreRetrieverMemory(
    retriever=retriever,
    memory_key="history"
)
```

6. Entity Memory

```
python
```

```
from langchain.memory import ConversationEntityMemory

memory = ConversationEntityMemory(
    llm=llm,
    return_messages=True
)

# Tracks entities (people, places, things) mentioned
```

7. LangGraph Persistence (Recommended)

```
python

from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import StateGraph

class ConversationState(TypedDict):
    messages: Annotated[list, operator.add]
    user_info: dict
    conversation_summary: str

memory = MemorySaver()

def conversation_node(state):
    # Access full history from state
    messages = state["messages"]
    response = llm.invoke(messages)
    return {"messages": [response]}

workflow = StateGraph(ConversationState)
workflow.add_node("chat", conversation_node)
workflow.set_entry_point("chat")
workflow.add_edge("chat", END)

app = workflow.compile(checkpointer=memory)

# Each thread maintains separate memory
config = {"configurable": {"thread_id": "user-123"}}
app.invoke({"messages": [HumanMessage(content="Hi")]}, config)
```

Tools & Function Calling

Creating Tools

1. @tool Decorator

```
python

from langchain_core.tools import tool

@tool
def search_api(query: str, max_results: int = 5) -> str:
    """Search the web for information.

    Args:
        query: The search query
        max_results: Maximum number of results to return
    """
    # Implementation
    return f"Results for {query}"

@tool
def calculator(expression: str) -> float:
    """Evaluate mathematical expressions.

    Args:
        expression: Math expression like "2 + 2" or "sqrt(16)"
    """
    return eval(expression)
```

2. StructuredTool

```
python
```

```

from langchain.tools import StructuredTool
from pydantic import BaseModel, Field

class SearchInput(BaseModel):
    query: str = Field(description="Search query")
    domain: str = Field(description="Domain to search in")

def search_function(query: str, domain: str) -> str:
    return f"Searching {domain} for {query}"

search_tool = StructuredTool.from_function(
    func=search_function,
    name="domain_search",
    description="Search within a specific domain",
    args_schema=SearchInput
)

```

3. BaseTool Class

```

python

from langchain.tools import BaseTool
from typing import Optional, Type

class CustomTool(BaseTool):
    name: str = "custom_calculator"
    description: str = "Performs custom calculations"

    def _run(self, expression: str) -> str:
        """Synchronous implementation"""
        result = eval(expression)
        return f"Result: {result}"

    async def _arun(self, expression: str) -> str:
        """Async implementation"""
        result = eval(expression)
        return f"Result: {result}"

```

Tool Collections

```

python

```

```
from langchain_community.tools import (
    WikipediaQueryRun,
    DuckDuckGoSearchRun,
    ArxivQueryRun,
    PubmedQueryRun,
    YouTubeSearchTool
)
from langchain_community.utilities import (
    WikipediaAPIWrapper,
    DuckDuckGoSearchAPIWrapper
)

# Wikipedia
wikipedia = WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper())

# Web Search
search = DuckDuckGoSearchRun(api_wrapper=DuckDuckGoSearchAPIWrapper())

# Scientific papers
arxiv = ArxivQueryRun()
pubmed = PubmedQueryRun()

# YouTube
youtube = YouTubeSearchTool()

tools = [wikipedia, search, arxiv, pubmed, youtube]
```

Toolkits

```
python
```

```
from langchain_community.agent_toolkits import (
    FileManagementToolkit,
    SQLDatabaseToolkit,
    create_python_agent
)
from langchain_community.utilities import SQLDatabase

# File management
file_toolkit = FileManagementToolkit(
    root_dir="./workspace",
    selected_tools=["read_file", "write_file", "list_directory"]
)

# SQL Database
db = SQLDatabase.from_uri("sqlite:///database.db")
sql_toolkit = SQLDatabaseToolkit(db=db, llm=llm)

# Python REPL
python_agent = create_python_agent(
    llm=llm,
    tool=PythonREPLTool(),
    verbose=True
)
```

Custom API Tools

```
python
```

```
import requests
```

```
@tool
```

```
def get_weather(city: str) -> str:
    """Get current weather for a city."""
    api_key = "your_key"
    url = f"https://api.openweathermap.org/data/2.5/weather"
    params = {"q": city, "appid": api_key}
    response = requests.get(url, params=params)
    data = response.json()
    return f"Weather in {city}: {data['weather'][0]['description']}"
```

```
@tool
```

```
def send_email(to: str, subject: str, body: str) -> str:
    """Send an email."""
    # Email sending logic
    return f"Email sent to {to}"
```

```
@tool
```

```
def create_calendar_event(title: str, date: str, time: str) -> str:
    """Create a calendar event."""
    # Calendar API logic
    return f"Event '{title}' created for {date} at {time}"
```

Tool Error Handling

```
python
```

```
from langchain_core.tools import ToolException

@tool
def risky_operation(value: str) -> str:
    """Operation that might fail."""
    try:
        result = perform_operation(value)
        return result
    except Exception as e:
        raise ToolException(f"Operation failed: {str(e)}")

# In agent
agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    handle_parsing_errors=True,
    max_iterations=5
)
```

Retrieval Systems (RAG)

Document Loading

```
python
```

```
from langchain_community.document_loaders import (
    PyPDFLoader,
    TextLoader,
    CSVLoader,
    UnstructuredMarkdownLoader,
    WebBaseLoader,
    DirectoryLoader,
    GitLoader,
    NotionDirectoryLoader
)
```

PDF

```
pdf_loader = PyPDFLoader("document.pdf")
pages = pdf_loader.load()
```

Web

```
web_loader = WebBaseLoader("https://example.com")
docs = web_loader.load()
```

Directory

```
loader = DirectoryLoader(
    "./docs",
    glob="**/*.md",
    loader_cls=UnstructuredMarkdownLoader
)
docs = loader.load()
```

Multiple sources

```
from langchain_community.document_loaders import UnstructuredFileLoader

loader = UnstructuredFileLoader("mixed_content.docx")
```

Text Splitting

python

```
from langchain.text_splitter import (
    RecursiveCharacterTextSplitter,
    CharacterTextSplitter,
    TokenTextSplitter,
    MarkdownTextSplitter,
    PythonCodeTextSplitter
)

# Recursive (recommended)
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    length_function=len,
    separators=["\n\n", "\n", " ", ""]
)

docs = text_splitter.split_documents(loaded_docs)

# Code-aware splitting
python_splitter = PythonCodeTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

# Semantic splitting
from langchain_experimental.text_splitter import SemanticChunker

semantic_splitter = SemanticChunker(
    embeddings,
    breakpoint_threshold_type="percentile"
)
```

Embeddings

```
python
```

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.embeddings import (
    HuggingFaceEmbeddings,
    CohereEmbeddings,
    BedrockEmbeddings
)

# OpenAI
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-large",
    dimensions=1024
)

# HuggingFace (local)
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

# Cohere
embeddings = CohereEmbeddings(model="embed-english-v3.0")

# Embed documents
embedded_docs = embeddings.embed_documents([doc.page_content for doc in docs])
# Embed query
embedded_query = embeddings.embed_query("What is AI?")
```

Vector Stores

```
python
```

```

from langchain_community.vectorstores import (
    Chroma,
    Pinecone,
    Weaviate,
    FAISS,
    Qdrant
)

# Chroma (local)
vectorstore = Chroma.from_documents(
    documents=docs,
    embedding=embeddings,
    persist_directory="./chroma_db"
)

# Pinecone (cloud)
from pinecone import Pinecone as PineconeClient

pc = PineconeClient(api_key="your_key")
index = pc.Index("index-name")

vectorstore = Pinecone.from_documents(
    docs, embeddings, index_name="index-name"
)

# FAISS (fast local)
vectorstore = FAISS.from_documents(docs, embeddings)
vectorstore.save_local("faiss_index")

# Load
vectorstore = FAISS.load_local("faiss_index", embeddings)

# Weaviate (scalable)
vectorstore = Weaviate.from_documents(
    docs,
    embeddings,
    client=weaviate_client,
    index_name="Document"
)

```

Retrieval Strategies

```
python
```

```
# Basic similarity search
retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 4}
)

# MMR (Maximum Marginal Relevance)
retriever = vectorstore.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 4, "fetch_k": 20, "lambda_mult": 0.5}
)

# Similarity with score threshold
retriever = vectorstore.as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={"score_threshold": 0.7, "k": 4}
)
```

Advanced Retrieval

Multi-Query Retriever

```
python

from langchain.retrievers.multi_query import MultiQueryRetriever

retriever = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(),
    llm=llm
)

# Generates multiple search queries automatically
docs = retriever.get_relevant_documents("What is machine learning?")
```

Contextual Compression

```
python
```

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import LLMChainExtractor

compressor = LLMChainExtractor.from_llm(llm)

compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=vectorstore.as_retriever()
)

# Returns only relevant parts of documents
docs = compression_retriever.get_relevant_documents("query")
```

Ensemble Retriever

```
python

from langchain.retrievers import EnsembleRetriever
from langchain_community.retrievers import BM25Retriever

# Combine vector search with keyword search
bm25_retriever = BM25Retriever.from_documents(docs)
bm25_retriever.k = 2

ensemble_retriever = EnsembleRetriever(
    retrievers=[bm25_retriever, vectorstore.as_retriever()],
    weights=[0.5, 0.5]
)
```

Parent Document Retriever

```
python
```

```
from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore

# Retrieve small chunks, return full documents
parent_splitter = RecursiveCharacterTextSplitter(chunk_size=2000)
child_splitter = RecursiveCharacterTextSplitter(chunk_size=400)

store = InMemoryStore()

retriever = ParentDocumentRetriever(
    vectorstore=vectorstore,
    docstore=store,
    child_splitter=child_splitter,
    parent_splitter=parent_splitter,
)

retriever.add_documents(docs)
```

RAG Chain

```
python
```

```

from langchain.chains import RetrievalQA
from langchain.chains.combine_documents import create_stuff_documents_chain

# Simple RAG
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=retriever,
    return_source_documents=True
)

result = qa_chain({"query": "What is AI?"})

# With custom prompt (LCEL)
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough

template = """Answer based on context:

Context: {context}

Question: {question}

Answer: """

prompt = ChatPromptTemplate.from_template(template)

def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)

rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

answer = rag_chain.invoke("What is machine learning?")

```

RAG with Citations

```
python
```

```
from langchain.chains import create_citation_fuzzy_match_chain
```

```
chain = create_citation_fuzzy_match_chain(llm)
```

```
context = "\n".join([doc.page_content for doc in docs])
```

```
answer = chain.run(question="What is AI?", context=context)
```

```
# Returns answer with citation markers [1], [2], etc.
```

Conversational RAG

```
python
```

```
from langchain.chains import create_history_aware_retriever
from langchain_core.prompts import MessagesPlaceholder

# Contextualize question based on history
contextualize_prompt = ChatPromptTemplate.from_messages([
    ("system", "Reformulate the question given chat history"),
    MessagesPlaceholder(variable_name="chat_history"),
    ("human", "{input}")
])

history_aware_retriever = create_history_aware_retriever(
    llm, retriever, contextualize_prompt
)

# QA chain with history
qa_prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer based on context:\n\n{context}"),
    MessagesPlaceholder(variable_name="chat_history"),
    ("human", "{input}")
])

from langchain.chains import create_retrieval_chain

rag_chain = create_retrieval_chain(
    history_aware_retriever,
    create_stuff_documents_chain(llm, qa_prompt)
)

# Use with memory
chat_history = []
response = rag_chain.invoke({
    "input": "What is AI?",
    "chat_history": chat_history
})

chat_history.extend([
    HumanMessage(content="What is AI?"),
    AIMessage(content=response["answer"])
])
```

Evaluation & Testing

Unit Testing

```
python

import pytest
from langchain.schema import HumanMessage

def test_agent_response():
    response = agent.invoke({"input": "Hello"})
    assert "messages" in response
    assert len(response["messages"]) > 0

def test_tool_execution():
    result = search_tool.run("AI news")
    assert len(result) > 0
    assert "AI" in result

@pytest.mark.asyncio
async def test_async_agent():
    response = await agent.ainvoke({"input": "Hello"})
    assert response is not None
```

Evaluation Metrics

```
python
```

```

from langchain.evaluation import (
    load_evaluator,
    EvaluatorType
)

# Criteria-based
criteria_eval = load_evaluator("criteria", criteria="conciseness")
result = criteria_eval.evaluate_strings(
    prediction="Long rambling answer...",
    input="What is 2+2?"
)

# String distance
distance_eval = load_evaluator("string_distance", distance="levenshtein")
result = distance_eval.evaluate_strings(
    prediction="The capital is Paris",
    reference="Paris is the capital"
)

# Embedding distance
embedding_eval = load_evaluator(
    "embedding_distance",
    embeddings=embeddings
)
result = embedding_eval.evaluate_strings(
    prediction="Machine learning uses algorithms",
    reference="ML utilizes algorithms"
)

# QA correctness
qa_eval = load_evaluator("qa")
result = qa_eval.evaluate_strings(
    input="What is the capital of France?",
    prediction="Paris",
    reference="The capital of France is Paris"
)

```

Custom Evaluators

```
python
```

```
from langchain.evaluation import StringEvaluator

class CustomEvaluator(StringEvaluator):
    @property
    def requires_input(self) -> bool:
        return True

    @property
    def requires_reference(self) -> bool:
        return True

    def _evaluate_strings(
        self,
        prediction: str,
        input: str = None,
        reference: str = None,
        **kwargs
    ) -> dict:
        # Your evaluation logic
        score = calculate_score(prediction, reference)
        return {
            "score": score,
            "reasoning": "Evaluation reasoning"
        }

evaluator = CustomEvaluator()
```

Batch Evaluation

```
python
```

```
from langsmith import evaluate

# Prepare test dataset
test_cases = [
    {"input": "What is 2+2?", "expected": "4"},
    {"input": "Capital of France?", "expected": "Paris"},
]

# Evaluator
def accuracy_evaluator(run, example):
    prediction = run.outputs.get("output", "")
    expected = example.outputs.get("expected", "")
    return {"score": 1.0 if prediction.strip() == expected.strip() else 0.0}

# Run evaluation
results = evaluate(
    lambda x: agent.invoke(x),
    data=test_cases,
    evaluators=[accuracy_evaluator],
    experiment_prefix="agent-accuracy-test"
)

print(f"Average accuracy: {results['results']['score'].mean()}")
```

A/B Testing

```
python
```

```
from langsmith import Client

client = Client()

# Test two different prompts
results_a = evaluate(
    lambda x: chain_a.invoke(x),
    data=dataset_name,
    experiment_prefix="prompt-a"
)

results_b = evaluate(
    lambda x: chain_b.invoke(x),
    data=dataset_name,
    experiment_prefix="prompt-b"
)

# Compare results in LangSmith UI
```

Synthetic Data Generation

```
python

from langchain.evaluation import QAGenerateChain

# Generate QA pairs from documents
qa_generator = QAGenerateChain.from_llm(llm)

qa_pairs = qa_generator.apply_and_parse(
    [{"doc": doc} for doc in docs]
)

# Use for testing
for pair in qa_pairs:
    print(f'Q: {pair['question']}')
    print(f'A: {pair['answer']}')
```

Production Deployment

API Deployment with FastAPI

```
python
```

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from langchain.callbacks import AsyncIteratorCallbackHandler
import asyncio

app = FastAPI()

class QueryRequest(BaseModel):
    message: str
    thread_id: str = "default"

class QueryResponse(BaseModel):
    response: str
    sources: list = []

@app.post("/chat", response_model=QueryResponse)
async def chat(request: QueryRequest):
    try:
        config = {"configurable": {"thread_id": request.thread_id}}

        result = await agent.ainvoke(
            {"messages": [HumanMessage(content=request.message)]},
            config=config
        )

        return QueryResponse(
            response=result["messages"][-1].content,
            sources=result.get("sources", [])
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# Streaming endpoint
@app.post("/chat/stream")
async def chat_stream(request: QueryRequest):
    async def event_generator():
        config = {"configurable": {"thread_id": request.thread_id}}

        async for chunk in agent.astream(
            {"messages": [HumanMessage(content=request.message)]},
            config=config
        ):
            if "messages" in chunk:
                yield f"data: {chunk['messages'][-1].content}\n\n"

    return StreamingResponse(

```

```
        event_generator(),
        media_type="text/event-stream"
    )

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Rate Limiting

```
python

from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

@app.post("/chat")
@limiter.limit("10/minute")
async def chat(request: QueryRequest):
    # Implementation
    pass
```

Caching

```
python
```

```
from langchain.cache import InMemoryCache, SQLiteCache
from langchain.globals import set_llm_cache
import redis
from langchain.cache import RedisCache

# In-memory cache
set_llm_cache(InMemoryCache())

# SQLite cache
set_llm_cache(SQLiteCache(database_path=".langchain.db"))

# Redis cache
redis_client = redis.Redis(host='localhost', port=6379)
set_llm_cache(RedisCache(redis_client))

# Semantic cache
from langchain.cache import RedisSemanticCache

set_llm_cache(
    RedisSemanticCache(
        redis_url="redis://localhost:6379",
        embedding=embeddings,
        score_threshold=0.9
    )
)
```

Error Handling & Retry Logic

```
python
```

```
from tenacity import (
    retry,
    stop_after_attempt,
    wait_exponential,
    retry_if_exception_type
)
from langchain.schema import OutputParserException

@retry(
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=1, min=4, max=10),
    retry=retry_if_exception_type(Exception)
)
async def invoke_with_retry(agent, input_data, config):
    try:
        return await agent.ainvoke(input_data, config)
    except OutputParserException:
        # Handle parsing errors
        pass
    except Exception as e:
        logger.error(f'Agent error: {e}')
        raise
```

Monitoring & Logging

python

```

import logging
from langchain.callbacks import get_openai_callback

# Setup logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# Track costs
with get_openai_callback() as cb:
    result = agent.invoke({"input": "Query"})
    logger.info(f'Tokens: {cb.total_tokens}, Cost: ${cb.total_cost}')

# Custom callback
from langchain.callbacks.base import BaseCallbackHandler

class CustomHandler(BaseCallbackHandler):
    def on_llm_start(self, serialized, prompts, **kwargs):
        logger.info(f'LLM started with prompts: {prompts}')

    def on_llm_end(self, response, **kwargs):
        logger.info(f'LLM finished: {response}')

    def on_tool_start(self, serialized, input_str, **kwargs):
        logger.info(f'Tool {serialized['name']} started')

    def on_agent_action(self, action, **kwargs):
        logger.info(f'Agent action: {action}')

agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    callbacks=[CustomHandler()]
)

```

Docker Deployment

dockerfile

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

yml

docker-compose.yml

version: '3.8'

services:

api:

build: .

ports:

- "8000:8000"

environment:

- OPENAI_API_KEY=\${OPENAI_API_KEY}

- LANGCHAIN_API_KEY=\${LANGCHAIN_API_KEY}

- LANGCHAIN_TRACING_V2=true

volumes:

- ./data:/app/data

depends_on:

- redis

redis:

image: redis:alpine

ports:

- "6379:6379"

Environment Management

python

```
from pydantic_settings import BaseSettings
```

```
class Settings(BaseSettings):
```

```
    openai_api_key: str
```

```
    anthropic_api_key: str
```

```
    langchain_api_key: str
```

```
    langchain_project: str = "production"
```

```
    redis_url: str = "redis://localhost:6379"
```

```
class Config:
```

```
    env_file = ".env"
```

```
settings = Settings()
```

Best Practices

1. Prompt Engineering

```
python
```

```
#  Good: Clear, specific instructions
```

```
prompt = """You are a helpful research assistant.
```

Task: Summarize the key findings from the provided research papers.

Guidelines:

- Focus on methodology and results
- Keep summaries under 200 words
- Include statistical significance when mentioned

Papers:

```
{documents}
```

```
Summary: """
```

```
#  Bad: Vague instructions
```

```
prompt = "Summarize these papers: {documents}"
```

2. Error Handling

```
python
```

```
#  Good: Comprehensive error handling
```

```
try:
```

```
    result = agent.invoke({"input": user_query})
```

```
except OutputParserException as e:
```

```
    logger.error(f"Parsing error: {e}")
```

```
    result = {"output": "I had trouble understanding the response. Please try again."}
```

```
except RateLimitError as e:
```

```
    logger.error(f"Rate limit: {e}")
```

```
    result = {"output": "System is busy. Please wait a moment."}
```

```
except Exception as e:
```

```
    logger.error(f"Unexpected error: {e}")
```

```
    result = {"output": "An error occurred. Our team has been notified."}
```

```
#  Bad: No error handling
```

```
result = agent.invoke({"input": user_query})
```

3. Token Management

```
python
```

```
#  Good: Track and limit tokens
```

```
from langchain.callbacks import get_openai_callback
```

```
with get_openai_callback() as cb:
```

```
    result = chain.invoke({"input": text})
```

```
    if cb.total_tokens > 50000:
```

```
        logger.warning(f"High token usage: {cb.total_tokens}")
```

```
    logger.info(f"Cost: ${cb.total_cost:.4f}")
```

```
# Use truncation for long texts
```

```
from langchain.text_splitter import TokenTextSplitter
```

```
splitter = TokenTextSplitter(chunk_size=3000, chunk_overlap=100)
```

```
chunks = splitter.split_text(long_text)
```

4. Security

```
python
```

 *Good: Input validation and sanitization*

```
from pydantic import BaseModel, validator
```

```
class UserQuery(BaseModel):
```

```
    text: str
```

```
    @validator('text')
```

```
    def validate_text(cls, v):
```

```
        if len(v) > 5000:
```

```
            raise ValueError("Query too long")
```

```
        if any(word in v.lower() for word in ["<script>", "DROP TABLE"]):
```

```
            raise ValueError("Invalid input")
```

```
        return v
```

API key management

```
import os
```

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
api_key = os.getenv("OPENAI_API_KEY")
```

Never hardcode keys

 *Bad*

```
api_key = "sk-abc123..."
```

5. Testing Strategy

```
python
```

```
# Unit tests
```

```
def test_agent_tool_calling():  
    response = agent.invoke({"input": "What's 5 + 3?"})  
    assert "8" in response["output"]
```

```
# Integration tests
```

```
def test_rag_pipeline():  
    docs = loader.load()  
    vectorstore = Chroma.from_documents(docs, embeddings)  
    retriever = vectorstore.as_retriever()  
    response = rag_chain.invoke("Test query")  
    assert len(response) > 0
```

```
# Evaluation tests
```

```
def test_response_quality():  
    evaluator = load_evaluator("criteria", criteria="helpfulness")  
    result = evaluator.evaluate_strings(  
        prediction=agent_response,  
        input=user_query  
    )  
    assert result["score"] > 0.7
```

6. Performance Optimization

```
python
```

```
#  Use async for concurrent operations
```

```
async def process_multiple_queries(queries):  
    tasks = [agent.ainvoke({"input": q}) for q in queries]  
    results = await asyncio.gather(*tasks)  
    return results
```

```
#  Batch processing
```

```
from langchain.chains import LLMChain
```

```
chain = LLMChain(llm=llm, prompt=prompt)  
results = await chain.abatch([{"input": q} for q in queries])
```

```
#  Streaming for better UX
```

```
async for chunk in agent.astream({"input": query}):  
    print(chunk, end="", flush=True)
```

7. Memory Management

```
python
```

```

#  Good: Clean up resources
from langchain.vectorstores import Chroma

vectorstore = Chroma(persist_directory="./db")
# Use vectorstore
vectorstore._client.persist() # Persist before closing
del vectorstore # Clean up

# Limit conversation history
def trim_messages(messages, max_tokens=4000):
    """Keep only recent messages within token limit"""
    total_tokens = 0
    trimmed = []

    for msg in reversed(messages):
        msg_tokens = len(msg.content) // 4 # Rough estimate
        if total_tokens + msg_tokens > max_tokens:
            break
        trimmed.insert(0, msg)
        total_tokens += msg_tokens

    return trimmed

```

8. Observability

```
python
```

 *Good: Comprehensive logging and tracing*

```
import structlog
```

```
logger = structlog.get_logger()
```

```
@traceable(name="process_query")
```

```
async def process_query(query: str, user_id: str):
```

```
    logger.info("query_started", user_id=user_id, query=query)
```

```
    try:
```

```
        result = await agent.ainvoke(
```

```
            {"input": query},
```

```
            config={
```

```
                "metadata": {
```

```
                    "user_id": user_id,
```

```
                    "timestamp": datetime.now().isoformat()
```

```
                },
```

```
                "tags": ["production", "user-query"]
```

```
            }
```

```
        )
```

```
    logger.info("query_completed", user_id=user_id, success=True)
```

```
    return result
```

```
except Exception as e:
```

```
    logger.error("query_failed", user_id=user_id, error=str(e))
```

```
    raise
```

9. Cost Optimization

python

```

#  Use smaller models when appropriate
from langchain_openai import ChatOpenAI

# For simple tasks
cheap_llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

# For complex reasoning
expensive_llm = ChatOpenAI(model="gpt-4", temperature=0)

# Route based on complexity
def get_llm_for_task(task_complexity: str):
    if task_complexity == "simple":
        return cheap_llm
    return expensive_llm

#  Implement caching
from langchain.cache import SQLiteCache
set_llm_cache(SQLiteCache(database_path=".cache.db"))

#  Use prompt compression
from langchain.prompts import PromptTemplate

# Compress context
compressed_prompt = PromptTemplate(
    template="Key points: {key_points}\nQuestion: {question}",
    input_variables=["key_points", "question"]
)

```

10. Version Control for Prompts

```
python
```

```
#  Good: Version and track prompts
```

```
PROMPTS = {  
    "v1": {  
        "template": "Answer: {input}",  
        "description": "Initial version"  
    },  
    "v2": {  
        "template": "Think step by step:\n{input}",  
        "description": "Added reasoning"  
    }  
}
```

```
def get_prompt(version="v2"):  
    return PromptTemplate(  
        template=PROMPTS[version]["template"],  
        input_variables=["input"]  
    )
```

```
# Track in LangSmith
```

```
chain = get_prompt("v2") | llm
```

```
result = chain.invoke(  
    {"input": query},  
    config={"tags": ["prompt-v2"]}  
)
```

Quick Reference Commands

Installation

```
bash
```

Core

```
pip install langchain langchain-community langchain-openai
```

Agents

```
pip install langgraph
```

Monitoring

```
pip install langsmith
```

Vector stores

```
pip install chromadb faiss-cpu pinecone-client
```

Utilities

```
pip install python-dotenv tiktoken
```

Environment Setup

```
bash
```

```
export OPENAI_API_KEY="sk-..."
```

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

```
export LANGCHAIN_TRACING_V2=true
```

```
export LANGCHAIN_API_KEY="ls-..."
```

```
export LANGCHAIN_PROJECT="my-project"
```

Common Imports

```
python
```

LLMs

```
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
```

Agents

```
from langgraph.graph import StateGraph, END
from langgraph.prebuilt import create_react_agent
from langchain.agents import AgentExecutor, create_openai_functions_agent
```

Tools

```
from langchain_core.tools import tool
from langchain.tools import StructuredTool
```

Memory

```
from langgraph.checkpoint.memory import MemorySaver
from langchain.memory import ConversationBufferMemory
```

RAG

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings
```

Monitoring

```
from langsmith import traceable, Client
```



Additional Resources

Official Documentation

- **LangChain:** <https://python.langchain.com>
- **LangGraph:** <https://langchain-ai.github.io/langgraph/>
- **LangSmith:** <https://docs.smith.langchain.com>

Community

- **Discord:** <https://discord.gg/langchain>
- **GitHub:** <https://github.com/langchain-ai/langchain>
- **Twitter:** @LangChainAI

Learning Resources

- **LangChain Academy:** <https://academy.langchain.com>

- LangChain Blog: <https://blog.langchain.dev>
 - Example Projects: <https://github.com/langchain-ai/langchain/tree/master/templates>
-

Common Patterns Cheatsheet

Task	Solution
Build simple chatbot	Use <code>ConversationChain</code> with <code>ConversationBufferMemory</code>
Build tool-using agent	Use <code>create_react_agent</code> with custom tools
Implement RAG	Load docs → Split → Embed → Store → Retrieve → Generate
Add memory to agent	Use LangGraph with <code>MemorySaver</code> checkpointer
Multi-step workflow	Use LangGraph <code>StateGraph</code> with conditional edges
Human approval needed	Use LangGraph with <code>interrupt_before</code>
Monitor production	Enable LangSmith tracing with metadata and tags
Evaluate performance	Use LangSmith <code>evaluate()</code> with custom evaluators
Handle long conversations	Use <code>ConversationSummaryMemory</code> or trim messages
Optimize costs	Cache results, use smaller models, compress prompts
Stream responses	Use <code>astream()</code> or <code>stream()</code> methods
Parallel execution	Use <code>RunnableParallel</code> or <code>asyncio.gather()</code>

Created with  for AI Agent Builders

Last Updated: February 2026